```
import pandas as pd
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, OneHotEncoder
from sklearn.compose import ColumnTransformer
from sklearn.metrics import mean_squared_error, r2_score
import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
import matplotlib.pyplot as plt
```

```
data = pd.read_csv("/content/AmesHousing.csv")
data.head()
```

	Order	PID	MS SubClass	MS Zoning	Lot Frontage	Lot Area	Street	Alley	Lot Shape	Land Contour	...	Pool Area	Pool QC	Fence	Misc Feature	Misc Val	M Sol
0	1	526301100	20	RL	141.0	31770	Pave	NaN	IR1	Lvl	...	0	NaN	NaN	NaN	0	
1	2	526350040	20	RH	80.0	11622	Pave	NaN	Reg	Lvl	...	0	NaN	MnPrv	NaN	0	
2	3	526351010	20	RL	81.0	14267	Pave	NaN	IR1	Lvl	...	0	NaN	NaN	Gar2	12500	
3	4	526353030	20	RL	93.0	11160	Pave	NaN	Reg	Lvl	...	0	NaN	NaN	NaN	0	
4	5	527105010	60	RL	74.0	13830	Pave	NaN	IR1	Lvl	...	0	NaN	MnPrv	NaN	0	

```
print("Dataset shape:", data.shape)
print("\nNumeric Features:", data.select_dtypes(include=[np.number]).columns.tolist())
print("\nCategorical Features:", data.select_dtypes(include=['object']).columns.tolist())
data.describe()
```

```
Dataset shape: (2930, 82)

Numeric Features: ['Order', 'PID', 'MS SubClass', 'Lot Frontage', 'Lot Area', 'Overall Qual', 'Overall Cond', 'Year Built', 'Yea

Categorical Features: ['MS Zoning', 'Street', 'Alley', 'Lot Shape', 'Land Contour', 'Utilities', 'Lot Config', 'Land Slope', 'Ne
```

	Order	PID	MS SubClass	Lot Frontage	Lot Area	Overall Qual	Overall Cond	Year Built	Year Remod/Add	Mas
count	2930.00000	2.930000e+03	2930.000000	2440.000000	2930.000000	2930.000000	2930.000000	2930.000000	2930.000000	2907.00
mean	1465.50000	7.144645e+08	57.387372	69.224590	10147.921843	6.094881	5.563140	1971.356314	1984.266553	101.89
std	845.96247	1.887308e+08	42.638025	23.365335	7880.017759	1.411026	1.111537	30.245361	20.860286	179.11
min	1.00000	5.263011e+08	20.000000	21.000000	1300.000000	1.000000	1.000000	1872.000000	1950.000000	0.00
25%	733.25000	5.284770e+08	20.000000	58.000000	7440.250000	5.000000	5.000000	1954.000000	1965.000000	0.00
50%	1465.50000	5.354536e+08	50.000000	68.000000	9436.500000	6.000000	5.000000	1973.000000	1993.000000	0.00
75%	2197.75000	9.071811e+08	70.000000	80.000000	11555.250000	7.000000	6.000000	2001.000000	2004.000000	164.00
max	2930.00000	1.007100e+09	190.000000	313.000000	215245.000000	10.000000	9.000000	2010.000000	2010.000000	1600.00

8 rows × 39 columns

```
numeric_features = data.select_dtypes(include=[np.number]).columns.tolist()
numeric_features.remove('SalePrice')
categorical_features = data.select_dtypes(include=['object']).columns.tolist()
data[numeric_features] = data[numeric_features].fillna(data[numeric_features].median())
data[categorical_features] = data[categorical_features].fillna("None")

preprocessor = ColumnTransformer(
    transformers=[
        ('num', StandardScaler(), numeric_features),
        ('cat', OneHotEncoder(handle_unknown='ignore'), categorical_features)
    ])

X = data.drop('SalePrice', axis=1)
y = data['SalePrice']
```

```
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42)

X_train = preprocessor.fit_transform(X_train)
X_test = preprocessor.transform(X_test)
```

```
model = Sequential([
    Dense(128, activation='relu', input_shape=(X_train.shape[1],)),
    Dropout(0.2),
    Dense(64, activation='relu'),
    Dense(32, activation='relu'),
    Dense(1)
])

model.compile(optimizer='adam', loss='mse', metrics=['mae'])

model.summary()
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:93: UserWarning: Do not pass an `input_shape`/`input_dim`
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

Model: "sequential"

Layer (type)	Output Shape	Param #
dense (Dense)	(None, 128)	40,704
dropout (Dropout)	(None, 128)	0
dense_1 (Dense)	(None, 64)	8,256
dense_2 (Dense)	(None, 32)	2,080
dense_3 (Dense)	(None, 1)	33

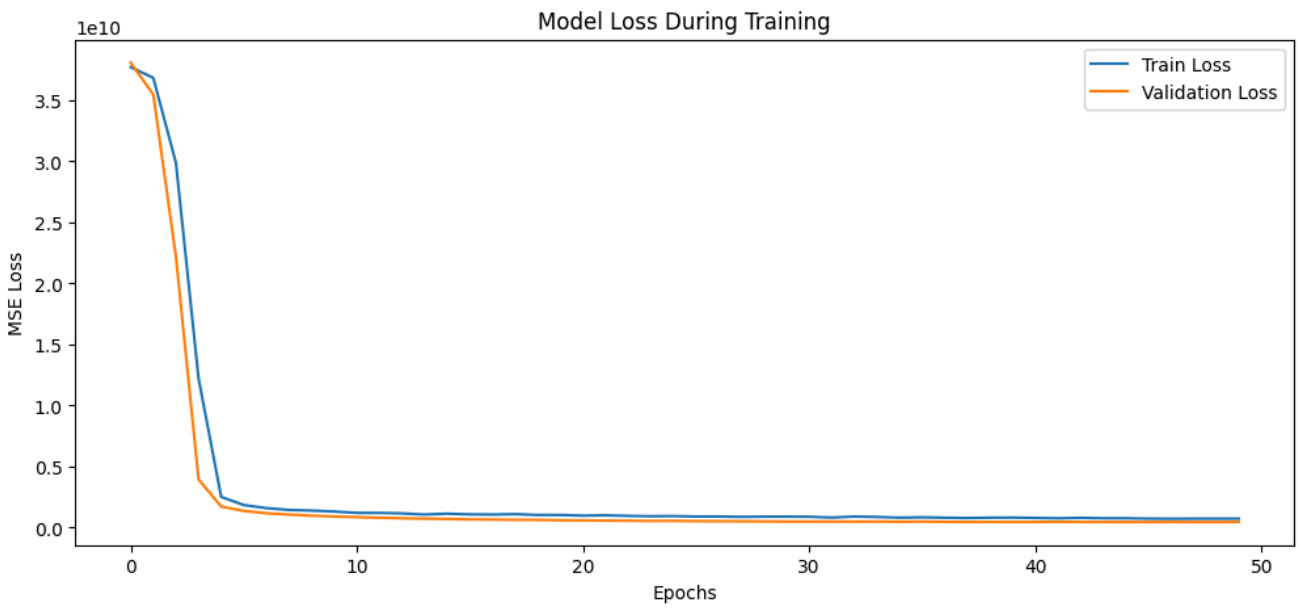
Total params: 51,073 (199.50 KB)
Trainable params: 51,073 (199.50 KB)
Non-trainable params: 0 (0.00 B)

```
history = model.fit(
    X_train, y_train,
    validation_split=0.2,
    epochs=50,
    batch_size=32,
    verbose=1
)

print("Model training completed.")
```

```
Epoch 38/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 784253440.0000 - mae: 17682.2168 - val_loss: 470951680.0000 - val_mae: 14890.82
Epoch 39/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 896526080.0000 - mae: 17409.8047 - val_loss: 472897888.0000 - val_mae: 14770.60
Epoch 40/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 786138560.0000 - mae: 16619.2148 - val_loss: 468625536.0000 - val_mae: 14949.34
Epoch 41/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 743605696.0000 - mae: 17225.3730 - val_loss: 470239328.0000 - val_mae: 14697.39
Epoch 42/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 6ms/step - loss: 708711168.0000 - mae: 16539.2910 - val_loss: 478007040.0000 - val_mae: 15263.19
Epoch 43/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 847963840.0000 - mae: 16793.5527 - val_loss: 470529600.0000 - val_mae: 14734.83
Epoch 44/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 570153152.0000 - mae: 15900.3184 - val_loss: 462291552.0000 - val_mae: 14674.06
Epoch 45/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 761290880.0000 - mae: 17122.4902 - val_loss: 461825184.0000 - val_mae: 14615.57
Epoch 46/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 796787584.0000 - mae: 16955.1621 - val_loss: 460352672.0000 - val_mae: 14603.33
Epoch 47/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step - loss: 900561536.0000 - mae: 17770.6934 - val_loss: 466023328.0000 - val_mae: 14740.14
Epoch 48/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 1s 8ms/step - loss: 871132544.0000 - mae: 17608.2500 - val_loss: 458307424.0000 - val_mae: 14630.40
Epoch 49/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 1s 10ms/step - loss: 778763008.0000 - mae: 16780.8105 - val_loss: 456305184.0000 - val_mae: 14543.5
Epoch 50/50
59/59 ━━━━━━━━━━━━━━━━━━━━ 1s 9ms/step - loss: 753394880.0000 - mae: 16449.7461 - val_loss: 469773888.0000 - val_mae: 14968.73
```

```
plt.figure(figsize=(12,5))
plt.plot(history.history['loss'], label='Train Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Model Loss During Training')
plt.xlabel('Epochs')
plt.ylabel('MSE Loss')
plt.legend()
plt.show()
```




```
y_pred = model.predict(X_test).flatten()

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)

print(f"Test RMSE: {rmse:.2f}")
print(f"Test R² Score: {r2:.2f}")
```

```
19/19 ━━━━━━━━━━━━━━━━━━━━ 0s 7ms/step
Test RMSE: 28852.45
Test R² Score: 0.90
```

```
comparison = pd.DataFrame({
    "Actual Price": y_test.values[:10],
    "Predicted Price": y_pred[:10]
})

comparison
```

	Actual Price	Predicted Price
0	161000	153717.515625
1	116000	116078.398438
2	196500	213546.484375
3	123600	123476.585938
4	126000	125736.023438
5	174190	193942.140625
6	200000	162080.093750
7	148500	149261.687500
8	88750	99657.687500
9	409900	354211.281250

```
plt.figure(figsize=(6,6))
plt.scatter(y_test, y_pred, alpha=0.5)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.title("Actual vs Predicted Housing Prices")

plt.plot(
    [y_test.min(), y_test.max()],
    [y_test.min(), y_test.max()]
)

plt.show()
```

```
single_house = X_test[0].reshape(1, -1)

predicted_price = model.predict(single_house)[0][0]
actual_price = y_test.iloc[0]
```

```
print(f"Predicted Price: {predicted_price:.2f}")
print(f"Actual Price: {actual_price:.2f}")
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 153ms/step
Predicted Price: 153717.53
Actual Price: 161000.00
```